Campus Facility Booking System

A Final-Year DBMS Project

PostgreSQL + FastAPI + Raw SQL

Contents

1 Introduction **1**
- 1.1 Technology Stack . . . 1
- 1.2 User Types and Sample Credentials . . . 1
  - 1.2.1 Sample Login Credentials . . . 2
- 1.3 Available Facilities . . . 2

2 Database Schema **3**
- 2.1 Layer 1: Identity and Finance . . . 3
  - 2.1.1 `Users` . . . 3
  - 2.1.2 `Students` and `Organizations` . . . 3
  - 2.1.3 `Wallet_Transactions` . . . 4
- 2.2 Layer 2: Inventory . . . 4
  - 2.2.1 `Facilities`, `Facility_Rooms`, `Facility_Slots` . . . 4
- 2.3 Layer 3: Booking Engine . . . 4
  - 2.3.1 `Bookings` and `Booking_Slots` . . . 4
- 2.4 Layer 4: Database-Level Enforcement (Triggers) . . . 5
  - 2.4.1 Trigger 1: `validate_ground_booking` . . . 5
  - 2.4.2 Trigger 2: `prevent_double_booking` . . . 6
- 2.5 Layer 5: Performance Indexing . . . 6
- 2.6 Concurrency Control: Serializability . . . 6

3 User Interface **7**
- 3.1 Login Screen . . . 7
- 3.2 Student Profile and Wallet . . . 7
- 3.3 Booking a Facility . . . 7
- 3.4 Wallet Transactions . . . 7
- 3.5 Admin: All Transactions . . . 7
- 3.6 Admin: Facility Utilization Report . . . 7
- 3.7 Admin: Facility Maintenance . . . 7

4 Testing and Verification **8**
- 4.1 ACID Coverage Map . . . 8
- 4.2 Concurrency Tests (Isolation) . . . 8
  - 4.2.1 `test_only_one_booking_wins` . . . 8
  - 4.2.2 `test_no_double_spend_on_shared_wallet` . . . 8

4.3 Rollback / Atomicity Tests 9
4.3.1 `test_rollback_on_insufficient_balance` 9
4.3.2 `test_rollback_on_ground_trigger` 9
4.3.3 `test_rollback_on_bad_slot_id` 9
4.3.4 `test_cancellation_refund_atomic` 10
4.4 Constraint Tests (Consistency) 10
4.5 Test Infrastructure 11

5 Conclusion **12**

1 Introduction

The **Campus Facility Booking System** is a final-year DBMS project that lets students, faculty, organizations, and administrators on a university campus book shared facilities such as seminar halls, labs, sports courts, guest rooms, and outdoor venues. The system is built around a wallet-based payment model, role-based access control, and strict ACID guarantees enforced at the database level.

The goal of the project is to demonstrate practical application of:

- Relational schema design with normalization up to **3NF**.
- ACID transactions with **atomicity**, **consistency**, **isolation**, and **durability**.
- Database-level integrity using `CHECK` constraints, `UNIQUE` indexes, foreign keys with `ON DELETE` policies, and PL/pgSQL triggers.
- Concurrency control with row-level locking (`SELECT FOR UPDATE`) and PostgreSQL `advisory locks` to enforce serializable behaviour for booking races.
- Indexed lookups on frequently-queried columns to keep response time low.

1.1 Technology Stack

Layer	Technology
Backend	Python 3.10+, FastAPI, psycopg2 (raw SQL, no ORM)
Database	PostgreSQL 16
Frontend	Vanilla HTML / CSS / JavaScript (single-page app)
Testing	pytest, httpx, ThreadPoolExecutor for concurrency

Table 1: Technology stack

The deliberate choice of **raw SQL over an ORM** is to keep the database logic visible and to allow direct demonstration of SQL features such as triggers, constraints, advisory locks, and partial unique indexes.

1.2 User Types and Sample Credentials

The system supports four user roles, distinguished by the `base_type` column in the `Users` table.

Role	Count (seeded)	Capabilities
Student	30	Login, top-up wallet, book most facilities, cancel own bookings, view own transactions.
Faculty	8	Same as students.
Organization	10	All of the above + can book restricted grounds (OAT, Pronite Ground).
Admin	2	All of the above + view all transactions across users, view facility utilization reports, take facilities offline.

Table 2: User roles and their capabilities

1.2.1 Sample Login Credentials

The seed script (`seed.py`) populates 50 users and writes `credentials.txt` with their email + password. Passwords follow the format **4 lowercase letters + 4 digits** (enforced by a database `CHECK` constraint).

Role	Email	Password (example)
Admin	`admin1@iitk.ac.in`	see `credentials.txt`
Admin	`admin2@iitk.ac.in`	see `credentials.txt`
Student	`aryan.maharaj91@iitk.ac.in`	see `credentials.txt`
Student	`ayushman.chander81@iitk.ac.in`	see `credentials.txt`
Organization	`antaragni@orgs.iitk.ac.in`	see `credentials.txt`
Faculty	any `*@faculty.iitk.ac.in`	see `credentials.txt`

Table 3: Sample login credentials

1.3 Available Facilities

The system models 15 distinct facilities across 7 types:

Type	Count	Examples
`Seminar_Hall`	2	Main Seminar Hall, North Block Seminar Hall
`Lab`	3	CS Lab 1, CS Lab 2, Electronics Lab
`SAC_Court`	3	Basketball, Badminton, Tennis
`OAT`	1	Open Air Theatre (Org-only)
`Pronite_Ground`	1	Pronite Main Ground (Org-only)
`Hall_Guest_Room`	4	Hall 1–4 Guest Rooms (with per-hall room numbers)
`Visitor_Hostel_Room`	1	Visitor Hostel (with rooms VH-101 through VH-120)

Table 4: Facility catalogue

Each facility has 8 recurring time slots (06:00 to 22:00, in 2-hour blocks) with prices ranging from Rs.0 (free, early morning) to Rs.500 (peak evening).

2 Database Schema

The schema is organized into five conceptual layers. All tables are in **Third Normal Form (3NF)**: every non-key attribute depends on the entire primary key and only the primary key. There are no transitive dependencies (e.g. student-specific data lives in `Students`, not in `Users`; organization-type lives in `Organizations`). This avoids update anomalies and keeps each fact stored in exactly one place.

2.1 Layer 1: Identity and Finance

2.1.1 Users

The single source of truth for any login. Each row carries shared columns; role-specific data lives in subtype tables (`Students`, `Organizations`).

```
CREATE TABLE Users (
    user_id        SERIAL PRIMARY KEY,
    name           VARCHAR(100) NOT NULL,
    email          VARCHAR(100) UNIQUE NOT NULL,
    password       VARCHAR(100) NOT NULL DEFAULT 'xxxx0000'
                   CHECK (password ~ '^[A-Za-z]{4}[0-9]{4}$'),
    base_type      VARCHAR(20)  NOT NULL
                   CHECK (base_type IN ('Student','Faculty','Admin','
                       Organization')),
    wallet_balance NUMERIC(10,2) NOT NULL DEFAULT 0.00
                   CHECK (wallet_balance >= 0),
    created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Constraints explained:

- `email UNIQUE` no two users can share an email.
- `password` regex check enforces format `[A-Za-z]{4}[0-9]{4}`.
- `base_type IN (...)` only the four allowed roles.
- `wallet_balance >= 0` invariant: wallet never goes negative, ever.

2.1.2 Students and Organizations

```
CREATE TABLE Students (
    user_id     INT PRIMARY KEY REFERENCES Users(user_id) ON DELETE CASCADE,
    roll_number VARCHAR(20) UNIQUE NOT NULL
);

CREATE TABLE Organizations (
    user_id  INT PRIMARY KEY REFERENCES Users(user_id) ON DELETE CASCADE,
    org_type VARCHAR(50) NOT NULL
             CHECK (org_type IN ('Club','Festival_Committee','Society'))
);
```

Foreign-key design: `user_id` is both the primary key and a foreign key to `Users`. Using `ON DELETE CASCADE` means deleting a user automatically removes their subtype row the subtype row exists *only* as a refinement of the user.

2.1.3 Wallet_Transactions

```
CREATE TABLE Wallet_Transactions (
    transaction_id   SERIAL PRIMARY KEY,
    user_id          INT NOT NULL REFERENCES Users(user_id) ON DELETE CASCADE,
    amount           NUMERIC(10,2) NOT NULL CHECK (amount > 0),
    transaction_type VARCHAR(20) NOT NULL
                     CHECK (transaction_type IN ('Deposit','Payment','Refund'))
                         ,
    description      TEXT,
    created_at       TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

This is an **immutable ledger** once a row is inserted, it is never updated. Together with the wallet-balance invariant, this guarantees auditability: the sum of `amount * sign(transaction_type)` over all rows for a user equals their current wallet balance.

2.2 Layer 2: Inventory

2.2.1 Facilities, Facility_Rooms, Facility_Slots

```
CREATE TABLE Facilities (
    facility_id    SERIAL PRIMARY KEY,
    name           VARCHAR(100) NOT NULL,
    type           VARCHAR(30) NOT NULL CHECK (type IN (...)),
    capacity       INTEGER NOT NULL CHECK (capacity > 0),
    is_operational BOOLEAN NOT NULL DEFAULT TRUE,
    created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Facility_Rooms (
    room_id        SERIAL PRIMARY KEY,
    facility_id    INT NOT NULL REFERENCES Facilities(facility_id) ON DELETE
        CASCADE,
    hall_number    INTEGER,
    room_number    VARCHAR(10) NOT NULL,
    is_operational BOOLEAN NOT NULL DEFAULT TRUE,
    UNIQUE (facility_id, hall_number, room_number),
    CHECK (hall_number IS NULL OR (hall_number BETWEEN 1 AND 14))
);

CREATE TABLE Facility_Slots (
    slot_id     SERIAL PRIMARY KEY,
    facility_id INT NOT NULL REFERENCES Facilities(facility_id) ON DELETE
        CASCADE,
    start_time  TIME NOT NULL,
    end_time    TIME NOT NULL,
    price       NUMERIC(10,2) NOT NULL DEFAULT 0.00 CHECK (price >= 0),
    CHECK (end_time > start_time),
    UNIQUE (facility_id, start_time, end_time)
);
```

Why three tables? Normalization. A facility has many rooms; a facility has a recurring weekly slot template. Storing room numbers or slots inside `Facilities` would require repeating the facility's name/type/capacity for every row, violating 2NF.

2.3 Layer 3: Booking Engine

2.3.1 Bookings and Booking_Slots

```
CREATE TABLE Bookings (
    booking_id   SERIAL PRIMARY KEY,
    user_id      INT NOT NULL REFERENCES Users(user_id) ON DELETE RESTRICT,
    booking_date DATE NOT NULL,
    total_cost   NUMERIC(10,2) NOT NULL CHECK (total_cost >= 0),
    status       VARCHAR(20) NOT NULL
                 CHECK (status IN ('Pending','Confirmed','Cancelled')),
    created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Booking_Slots (
    booking_slot_id SERIAL PRIMARY KEY,
    booking_id      INT NOT NULL REFERENCES Bookings(booking_id) ON DELETE
        CASCADE,
    slot_id         INT NOT NULL REFERENCES Facility_Slots(slot_id) ON DELETE
        RESTRICT,
    room_id         INT REFERENCES Facility_Rooms(room_id) ON DELETE RESTRICT,
    price_charged   NUMERIC(10,2) NOT NULL CHECK (price_charged >= 0)
);

CREATE UNIQUE INDEX uq_booking_slots_with_room
    ON Booking_Slots (booking_id, slot_id, room_id) WHERE room_id IS NOT NULL;
CREATE UNIQUE INDEX uq_booking_slots_no_room
    ON Booking_Slots (booking_id, slot_id) WHERE room_id IS NULL;
```

Key design choices:

- `ON DELETE RESTRICT` on `Bookings.user_id` prevents deleting a user who has booking history. Bookings are an audit record.
- `ON DELETE CASCADE` on `Booking_Slots.booking_id` line items only exist as part of a booking; if the booking is hard-deleted (rare; we usually `Cancel` instead), the line items go with it.
- Two **partial unique indexes** on `Booking_Slots` prevent the same slot appearing twice in one cart, with NULL-aware semantics. Standard `UNIQUE` treats two NULL `room_ids` as distinct, which would let duplicates slip through; partial indexes split the world into "with room" and "without room" cases.

2.4 Layer 4: Database-Level Enforcement (Triggers)

2.4.1 Trigger 1: `validate_ground_booking`

Only `Organization` users can book OAT or Pronite Ground. Enforced as a `BEFORE INSERT/UPDATE` trigger on `Booking_Slots`.

```
CREATE OR REPLACE FUNCTION validate_ground_booking()
RETURNS TRIGGER AS $$
DECLARE u_base_type VARCHAR; fac_type VARCHAR;
BEGIN
    SELECT f.type INTO fac_type
      FROM Facilities f JOIN Facility_Slots fs ON f.facility_id = fs.
          facility_id
     WHERE fs.slot_id = NEW.slot_id;
    SELECT u.base_type INTO u_base_type
      FROM Users u JOIN Bookings b ON u.user_id = b.user_id
     WHERE b.booking_id = NEW.booking_id;
    IF fac_type IN ('OAT','Pronite_Ground') AND u_base_type <> 'Organization'
        THEN
        RAISE EXCEPTION 'Access␣Denied:␣Only␣official␣Organizations␣can␣book␣
            major␣grounds.';
```

```
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

2.4.2 Trigger 2: prevent_double_booking

The most important integrity rule: no two bookings can hold the same `slot_id` on the same `booking_date` (cancelled bookings are ignored). Combined with application-side advisory locks, this gives serializable booking behaviour.

```
CREATE OR REPLACE FUNCTION prevent_double_booking()
RETURNS TRIGGER AS $$
DECLARE new_date DATE; conflict INT;
BEGIN
    SELECT booking_date INTO new_date FROM Bookings WHERE booking_id = NEW.
        booking_id;
    SELECT COUNT(*) INTO conflict
      FROM Booking_Slots bs JOIN Bookings b ON b.booking_id = bs.booking_id
     WHERE bs.slot_id = NEW.slot_id AND b.booking_date = new_date
       AND b.status IN ('Pending','Confirmed') AND bs.booking_id <> NEW.
           booking_id
       AND COALESCE(bs.room_id, -1) = COALESCE(NEW.room_id, -1);
    IF conflict > 0 THEN
        RAISE EXCEPTION 'Slot␣%␣on␣%␣is␣already␣booked.', NEW.slot_id, new_date
            ;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

2.5 Layer 5: Performance Indexing

Indexes accelerate the most frequent queries the API issues:

2.6 Concurrency Control: Serializability

PostgreSQL's default `READ COMMITTED` isolation level is too weak for the booking workflow two concurrent transactions can both see "no conflict" because neither has committed yet. Three layers ensure correct serializable behaviour:

1. **Advisory locks (application layer)**: `create_booking` calls `pg_advisory_xact_lock(slot_id, date_int)` before any inserts. Concurrent requests for the same slot+date queue at this lock; the second only proceeds after the first commits or rolls back.

2. **Row-level locks (financial layer)**: `SELECT ... FOR UPDATE` on the user's row before reading the wallet balance. This serializes wallet-spending decisions for the same user.

3. **Trigger-based check (defense in depth)**: even if locks failed, the `prevent_double_booking` trigger sees committed conflicts and raises an exception, rolling back the transaction.

3 User Interface

This section presents screenshots of the major screens. The application is a single-page app served from `static/index.html`, communicating with the FastAPI backend over JSON.

3.1 Login Screen

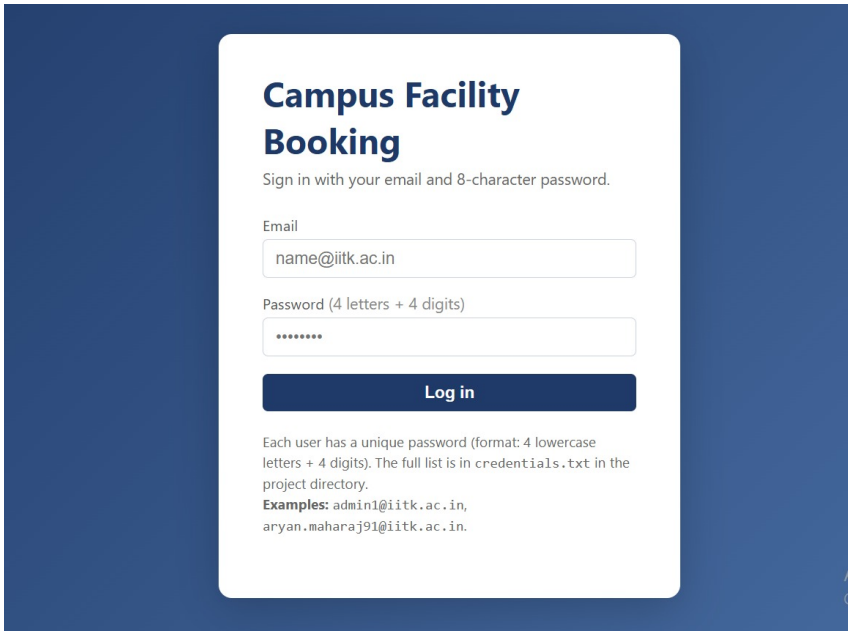

Figure 1: Login screen. Users authenticate with their campus email and an 8-character password (4 letters + 4 digits, enforced by a `CHECK` constraint).

3.2 Student Profile and Wallet

3.3 Booking a Facility

3.4 Wallet Transactions

3.5 Admin: All Transactions

3.6 Admin: Facility Utilization Report

3.7 Admin: Facility Maintenance

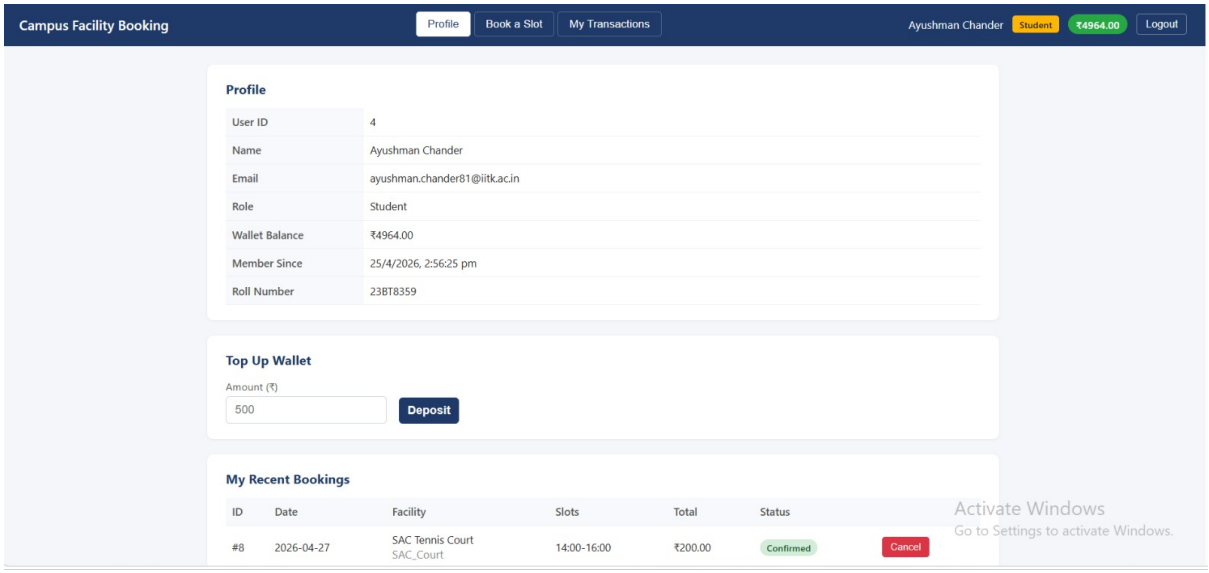

Figure 2: Student profile view showing user details, wallet balance, top-up form, and recent bookings. The wallet badge in the header is refreshed on every navigation to keep the displayed balance in sync with the database.

4 Testing and Verification

The project includes both a **manual UI testing plan** (`TESTING.md`) and an **automated pytest suite** of 17 tests covering all four ACID properties.

4.1 ACID Coverage Map

4.2 Concurrency Tests (Isolation)

4.2.1 `test_only_one_booking_wins`

Purpose: Prove that exactly 1 of N concurrent bookings of the same slot succeeds.

Setup: 30 student bots are loaded from `credentials.txt`, each with $\geq$ Rs.1000 wallet balance. The first available slot on a target facility/date is identified.

Action: Using `ThreadPoolExecutor(max_workers=30)`, all 30 bots fire `POST /api/bookings` for that slot at the same instant.

Expected: 1 winner (HTTP 201). 29 losses (HTTP 400 with "Slot X on date is already booked"). DB query confirms exactly 1 live `Booking_Slots` row.

What it proves: Without protection, two transactions in `READ COMMITTED` could both pass the `prevent_double_booking` trigger check (since neither sees the other's uncommitted insert) and both succeed. The combination of `pg_advisory_xact_lock` (serializing access) and the trigger (rejecting confirmed conflicts) guarantees exactly one winner.

4.2.2 `test_no_double_spend_on_shared_wallet`

Purpose: Prove that `SELECT FOR UPDATE` prevents over-spending under concurrency.

Setup: One student's wallet is reset to exactly Rs.500. Twenty distinct Rs.200 slots (across multiple non-restricted facilities) are picked. Twenty separate `Bot` instances all log in as the same student.

Action: 20 concurrent `POST /api/bookings` requests are fired in parallel.

Expected: Exactly $\lfloor 500/200 \rfloor = 2$ bookings succeed; 18 fail with "insufficient balance"; final wallet balance is exactly Rs.100; `Wallet_Transactions` payment count equals winners.

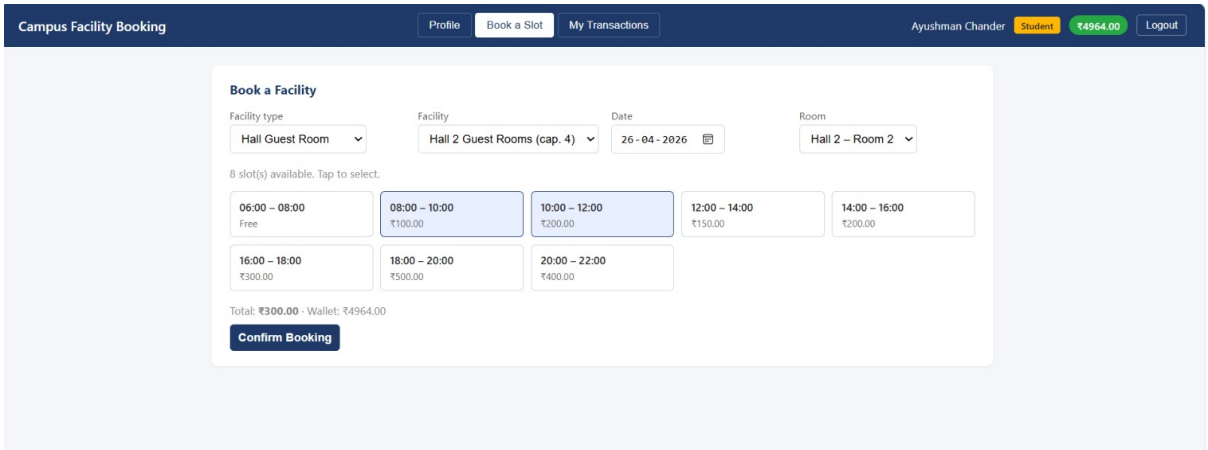

Figure 3: The booking flow. The user picks a facility type, a specific facility, a date, and (for hall guest rooms or the visitor hostel) a room. Available slots are computed server-side by excluding any slot already in a Pending or Confirmed booking on that date. Selected slots are highlighted in blue; the running total updates live.

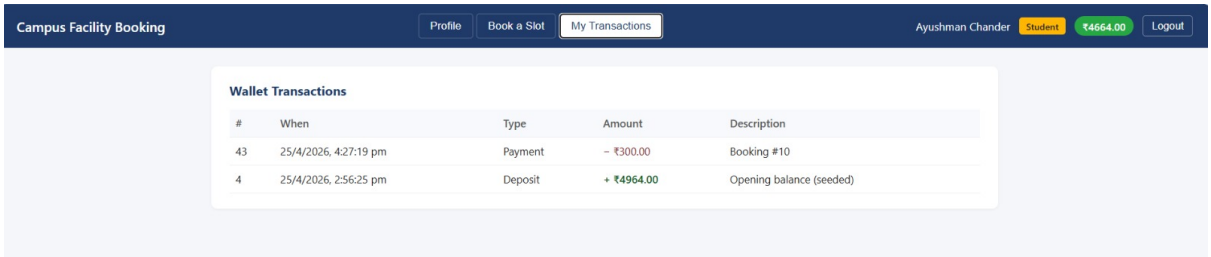

Figure 4: The user's wallet transaction ledger every Deposit, Payment, and Refund. Sorted newest-first using the `idx_wallet_tx_user_time` index.

What it proves: The `SELECT user_id, wallet_balance FROM Users WHERE user_id = %s FOR UPDATE` statement acquires a row-level exclusive lock on the user row. Concurrent transactions block at this lock and re-read the now-debited wallet, ensuring `wallet_balance` never goes negative.

4.3 Rollback / Atomicity Tests

4.3.1 `test_rollback_on_insufficient_balance`

A student with Rs.10 attempts to book a Rs.500 slot. The endpoint snapshots `(wallet, count(bookings), count(transactions))` before and after. Assertion: pre-state and post-state are byte-identical. The `conn.rollback()` on the failure path undoes the booking-header insert, so no orphan rows survive.

4.3.2 `test_rollback_on_ground_trigger`

A student with Rs.10000 (so balance is not the failure cause) tries to book OAT. The `validate_ground_booking` trigger raises mid-transaction; everything inserted before the trigger fires (the `Bookings` header) rolls back.

4.3.3 `test_rollback_on_bad_slot_id`

Sending `slot_ids: [999_999]` causes a 400 "invalid slot ID". Verifies validation errors that fire before any insert leave the DB clean.

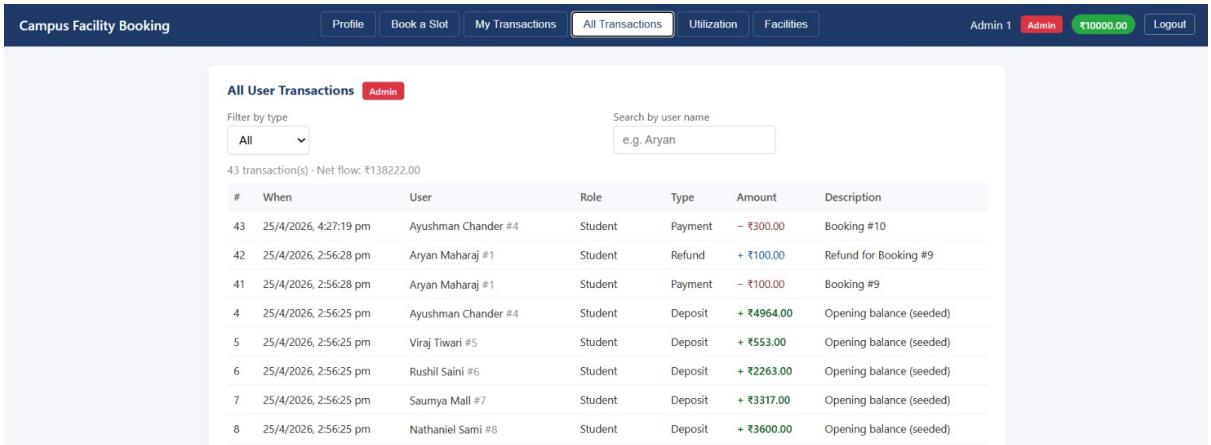

Figure 5: Admin-only "All Transactions" view. Lists every wallet transaction across every user, with filters by transaction type and user name. Net flow at the top of the table aggregates Deposits + Refunds minus Payments.

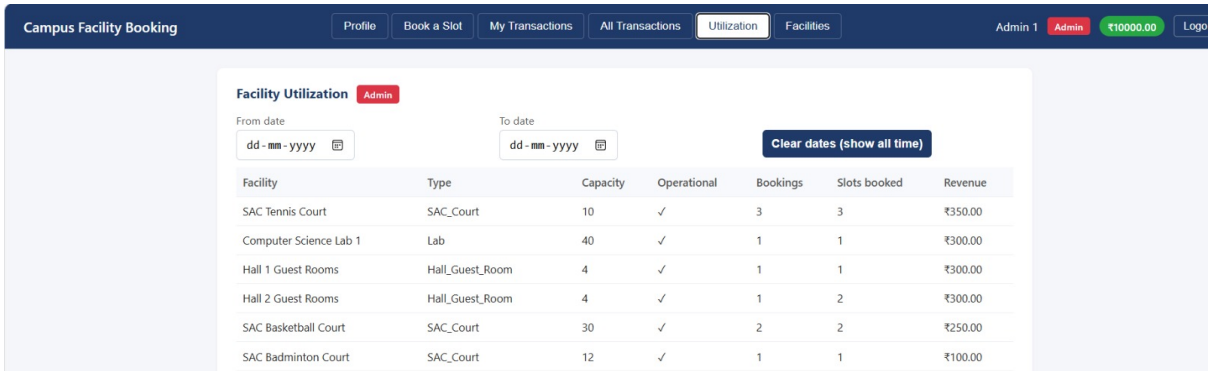

Figure 6: Admin "Utilization" report. Shows bookings, slots booked, and revenue per facility backed by a CTE that joins `Bookings`, `Booking_Slots`, `Facility_Slots`, and `Facilities` with optional date-range filters. Sorted by revenue descending.

4.3.4 `test_cancellation_refund_atomic`

Books a paid slot, then immediately cancels. Verifies: net wallet change = 0, bookings count +1 (cancelled but kept for history), transactions count +2 (one Payment + one Refund). The cancel endpoint touches three tables in one atomic commit.

4.4 Constraint Tests (Consistency)

These hit PostgreSQL directly (bypassing the API) to verify every `CHECK`, `UNIQUE`, foreign-key, and trigger fires when violated.

#	Test	What it verifies
9.1	`password_format_check`	Password regex `[A-Za-z]{4}[0-9]{4}`.
9.2	`wallet_balance_never_negative`	`wallet_balance >= 0`.
9.3	`base_type_check`	`base_type IN ('Student','Faculty','Admin','Organization')`.
9.4	`email_unique`	`Users.email UNIQUE`.
9.5	`wallet_txn_amount_positive`	`Wallet_Transactions.amount > 0`.
9.6	`facility_slot_endtime_after_start`	`end_time > start_time`.

#	Test	What it verifies
9.7	`facility_slot_unique`	`UNIQUE(facility_id, start_time, end_time)`.
9.8	`double_booking_trigger`	`prevent_double_booking` fires on duplicate slot+date.
9.9	`ground_booking_trigger`	Trigger blocks a Student from booking OAT.
9.10	`org_CAN_book_OAT`	Counter-test: Organization succeeds where Student failed.
9.11	`delete_user_with_bookings_restricted`	`ON DELETE RESTRICT` on `Bookings.user_id`.

Table 7: Every constraint test in test_constraints.py

4.5 Test Infrastructure

Bot harness (`tests/bots.py`): wraps `httpx.Client` for one logged-in user, exposing `login / profile / deposit / availability / book / cancel`. `load_bots(n, role)` returns N pre-authenticated bots.

Fixtures (`tests/conftest.py`):

- `api_url` confirms the backend is reachable via `/api/health`; skips tests if not.
- `fresh_db` truncates only the transactional tables (`Booking_Slots`, `Bookings`, `Wallet_Transactions`) and resets every wallet to Rs.1000. Users and facilities are preserved across runs.
- `db_conn` a direct `psycopg2` connection in a transaction that's always rolled back at teardown. Used by constraint tests.

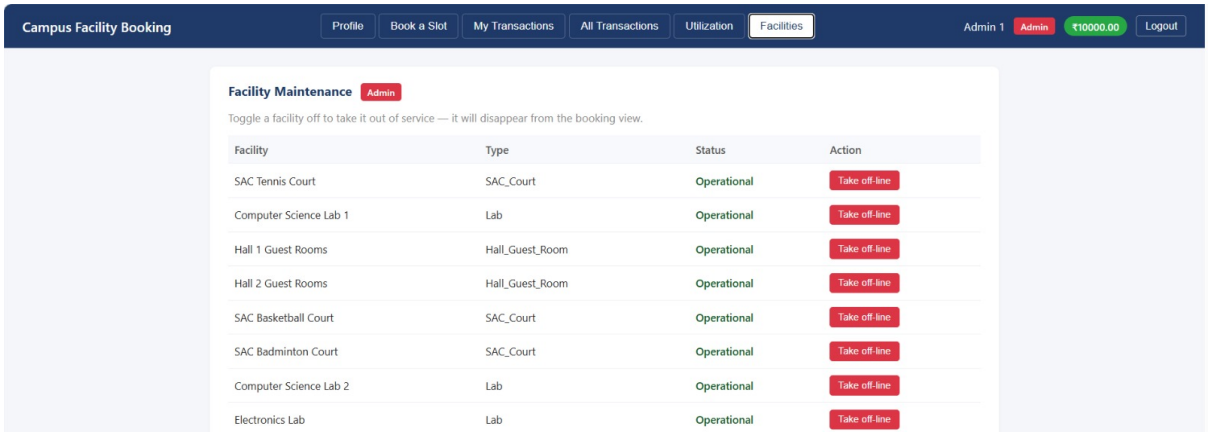

Figure 7: Admin "Facilities" maintenance view. Toggling a facility off-line sets `is_operational = FALSE`, which immediately removes it from the booking dropdown for non-admin users (the `/api/facilities` endpoint filters on this column).

5 Conclusion

The Campus Facility Booking System demonstrates a database-first design philosophy: as much of the business logic as possible lives in the database (constraints, triggers, indexes), with the application layer responsible only for serializing access and exposing the API.

Key takeaways:

- **Schema in 3NF** no redundant attributes, every fact in exactly one place.
- **ACID guarantees** are not optional every test in the suite exercises one or more of A, C, I, D.
- **Concurrency cannot rely on triggers alone** `READ COMMITTED` permits the trigger check to pass for both racers. Advisory locks plus row-level `FOR UPDATE` locks fill that gap and produce serializable behaviour.
- **Partial unique indexes with NULL-aware predicates** elegantly solve the "same slot in multiple lines of one cart" duplication problem when `room_id` is optional.
- **Audit invariants** are upheld at the database level, not by hopeful application code: `wallet_balance >= 0`, `amount > 0`, `ON DELETE RESTRICT` on bookings.

The system has been validated through:

- 17 automated pytest tests covering concurrency, rollback, and constraint scenarios.
- Manual UI walkthroughs documented in `TESTING.md`.
- End-to-end flows shown in the screenshots above.

The codebase is intentionally minimal (~600 lines of Python + ~250 lines of SQL + a single HTML file) to keep the database layer the centerpiece of the design.

Source code: github.com/be-water22/facility-booking

```
============================================= test session starts =============================================
platform win32 -- Python 3.11.1, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\rythm\Documents\facility-booking-main\venv\S
cripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\rythm\Documents\facility-booking-main
plugins: anyio-4.13.0, Faker-40.15.0
collected 17 items

tests/test_concurrency_booking.py::test_only_one_booking_wins PASSED                                    [  5%]
tests/test_concurrency_wallet.py::test_no_double_spend_on_shared_wallet PASSED                          [ 11%]
tests/test_constraints.py::test_password_format_check PASSED                                            [ 17%]
tests/test_constraints.py::test_wallet_balance_never_negative PASSED                                    [ 23%]
tests/test_constraints.py::test_base_type_check PASSED                                                  [ 29%]
tests/test_constraints.py::test_email_unique PASSED                                                     [ 35%]
tests/test_constraints.py::test_wallet_txn_amount_positive PASSED                                       [ 41%]
tests/test_constraints.py::test_facility_slot_endtime_after_start PASSED                                [ 47%]
tests/test_constraints.py::test_facility_slot_unique PASSED                                             [ 52%]
tests/test_constraints.py::test_double_booking_trigger PASSED                                           [ 58%]
tests/test_constraints.py::test_ground_booking_trigger_blocks_non_org PASSED                            [ 64%]
tests/test_constraints.py::test_org_CAN_book_OAT PASSED                                                 [ 70%]
tests/test_constraints.py::test_delete_user_with_bookings_is_restricted PASSED                          [ 76%]
tests/test_rollback.py::test_rollback_on_insufficient_balance PASSED                                    [ 82%]
tests/test_rollback.py::test_rollback_on_ground_trigger PASSED                                          [ 88%]
tests/test_rollback.py::test_rollback_on_bad_slot_id PASSED                                             [ 94%]
tests/test_rollback.py::test_cancellation_refund_atomic PASSED                                          [100%]

========================================== 17 passed in 195.35s (0:03:15) ==========================================
```

Figure 8: All 17 automated tests passing in 195 seconds. Four test files cover concurrency, constraints, rollback, and the wallet double-spend scenario.

Index	Table	Why
`idx_facility_slots_facility`	`Facility_Slots(facility_id)`	Availability lookup is always filtered by facility.
`idx_bookings_date_status`	`Bookings(booking_date, status)`	Admin utilization report and availability checks scan by date+status.
`idx_booking_slots_slot`	`Booking_Slots(slot_id)`	The double-booking trigger queries Booking_Slots by slot_id without this, every insert would scan the table.
`idx_wallet_tx_user_time`	`Wallet_Transactions(user_id, created_at DESC)`	"My Transactions" page sorts a user's

ACID Property	Test File	Number of Tests
Atomicity	`test_rollback.py`	4
Consistency	`test_constraints.py`	11
Isolation	`test_concurrency_booking.py`, `test_concurrency_wallet.py`	2
Durability	All 17 tests (every assertion reads back from DB)	implicit

Table 6: Mapping of automated tests to ACID properties